

Universidad de Valladolid

Cuaderno de bitácora ESI - Grupo 16

Ingeniería informática

TI

Álvaro López Primo
Edgar Santiesteban Mendo
Marcos Álvarez Fulconis

Curso 25-26

Índice

1. Introducción **3**

2. Primeros pasos **3**
2.1. Acceso y configuración inicial a la máquina . . . 3
2.2. Configuración y acceso de grafana . . . 3
2.3. Elección de bases de datos . . . 4
2.3.1. Relacionales vs no relacional . . . 4
2.3.2. ¿Qué sistemas de gestión de bases de datos vamos a utilizar? . . . 4
2.4. Lenguaje de programación del script . . . 5
2.5. Creación del usuario para la bbdd . . . 5
2.6. Elección de las métricas . . . 6

3. Base de datos en Postgressql **9**
3.1. Agrupación de datos y estructura de tablas . . . 9
3.2. Gestión del almacenamiento y particionamiento . . . 11
3.2.1. Automatización del proceso . . . 11
3.3. Explicación de los scripts sobre SQL . . . 12

4. Conexión con grafana **13**
4.1. Permisos en PostgreSQL para el usuario `grafana` . . . 13
4.1.1. Acceso a la base de datos . . . 13
4.1.2. Permisos de Lectura (Para el Dashboard de Grafana) . . . 13
4.1.3. Permisos de Escritura (Para el Script de Python) . . . 13

5. Convertir el Script en un Daemon (Servicio de Systemd) **13**
5.1. Creación del archivo de configuración . . . 13
5.2. Activación y ejecución del servicio . . . 14

6. Códigos Python **14**
6.1. Módulo de recolección de métricas (Collectors) . . . 14
6.1.1. Diseño general . . . 14
6.1.2. Collector de uso de CPU por núcleo . . . 14
6.1.3. Collector de métricas de rendimiento global . . . 15
6.1.4. Collector de procesos más relevantes . . . 15
6.1.5. Collector de almacenamiento . . . 16
6.1.6. Collector de estado del sistema . . . 16
6.2. Módulo de obtención de métricas (Metrics) . . . 17
6.2.1. Diseño general . . . 17
6.2.2. Métricas de CPU . . . 17
6.2.3. Métricas de memoria . . . 17
6.2.4. Métricas de red . . . 18
6.2.5. Métricas de almacenamiento . . . 18
6.2.6. Métricas de procesos . . . 18
6.2.7. Métricas del sistema . . . 19
6.3. Módulo de persistencia de datos . . . 19
6.3.1. Gestión de conexiones . . . 19
6.3.2. Inserción de métricas . . . 20
6.4. Módulo de configuración . . . 20
6.4.1. Funcionamiento . . . 20
6.5. Módulo de planificación (Scheduler) . . . 21
6.5.1. Funcionamiento . . . 21
6.6. Módulo principal de ejecución . . . 22

6.6.1. Funcionamiento 22

7. Métricas de Grafana **23**
7.1. Memoria 23
7.1.1. SWAP-CACHÉ-RAM 24
7.1.2. Métrica de la RAM que consumen ciertos procesos 24
7.2. Almacenamiento 25
7.3. CPU 25
7.3.1. Uso de la CPU 25
7.3.2. Uso de los *cores* 26
7.4. Procesos 26
7.4.1. Número y tipos ejecutándose 26
7.5. *Network* 27
7.5.1. Tráfico entrante y saliente 27
7.5.2. Paquetes perdidos 27

8. Reparto del trabajo **28**

1. Introducción

El objetivo de esta práctica es generar un código para poder garantizar que el sistema funcione de manera correcta, aplicando métricas del rendimiento para saber cuándo la máquina se encuentra en un estado crítico, por una carga de trabajo elevada o por problemas como cuellos de botella.
Usaremos este documento para ir reflejando el trabajo de los diferentes integrantes de este grupo, desde el acceso inicial a las máquinas, hasta la interpretación de las métricas del sistema.

2. Primeros pasos

2.1 Acceso y configuración inicial a la máquina

Lo primero que hicimos fue acceder por ssh a la máquina, debido a que es un entorno de la escuela, estamos forzados a emplear la VPN de la escuela para poder conectarnos desde eduroam.

```
1            ssh -p 31161 usuario@virtual.lab.inf.uva.es
2
```

Listing 1: Comando ssh para el acceso remoto a la máquina virtual

Una vez hecho esto, procedemos a cambiar la contraseña por defecto de las máquinas y establecemos una nueva, **akelarre**.
Además comprobamos las especificaciones de nuestra máquina.

```
1            lscpu   #CPU
2            free -h  #RAM
3            sudo lshw -short #Indice hardware
4            lsblk #discos duros y particiones
5
```

Listing 2: Comandos para comprobar el *hardware* de la máquina virtual

2.2 Configuración y acceso de grafana

Una vez ya tenemos acceso a la máquina y la hemos s̈ecurizado; procedemos a instalar grafana, el cual es un *software* libre para visualizar datos, es empleado en el mundo real para monitoreo de nuestro computador.

```
1        sudo apt-get install -y adduser libfontconfig1 musl
2        wget https://dl.grafana.com/grafana-enterprise/release/12.4.1/grafana-
    enterprise_12.4.1_22846628243_linux_amd64.deb
3        sudo dpkg -i grafana-enterprise_12.4.1_22846628243_linux_amd64.deb
4
```

Listing 3: Comandos para instalar grafana

Una vez que tenemos esto configurado, procedemos a comprobar si el daemon de grafana está funcionando

```
1        sudo systemctl status grafana-server
2        sudo systemctl enable grafana-server #Permite que se inicie el daemon en el
     siguiente reinicio
3        sudo systemctl start grafana-server #Permite iniciar el daemon
4
```

Listing 4: Comando para verificar el estado del *daemon* de grafana

Acceso a Grafana desde URL de la máquina virtual con contraseña y usuario original siendo: admin. Ha sido modificada la contraseña posteriormente a **akelarre**. Por tanto se nos ha quedado en:

Link URL Grafana: **http://virtual.lab.inf.uva.es:31162**
Usuario: **admin**
Contrseña: **akelarre**

2.3 Elección de bases de datos

En este punto, lo que vamos a determinar es la tecnología de bases de datos para poder almacenar los diferentes datos y métricas que vayamos recolectando, como en el enunciado no se nos obliga a utilizar una en concreto, empezamos a discutir qué bbdd era mejor

2.3.1 Relacionales vs no relacional

Comenzamos planteándonos qué tipo de bbdd debíamos emplear, los dos grandes paradigmas son relacionales, las cuales son muy rígdas con una serie de filas o columnas cuyos elemento no deben ser nulos, mientras que el otro tipo, no son realmente tablas sino más bien colecciones y que en vez de trabajar con *JOIN* trabaja con referencias.
Debido a la poca experiencia que tenemos con bases de datos, nos hemos decantado por las relacionales, que son las clásicas, y las que hemos usado en la asignatura de ADBD.

2.3.2 ¿Qué sistemas de gestión de bases de datos vamos a utilizar?

Vamos a partir de la base de que nuestra VM tiene recursos limitados de RAM, CPU y almacenamiento, por lo que esta decisión es clave, los gestores que hemos manejado son *PosgreSQL, MariaDB, OracleSQLPlus*, este último siendo el que ya conocemos los 3 del grupo.
Tras una búsqueda en internet, y hacer una contraposición de puntos, hemos optado por **PostgreSQL** utilizando un enfoque ***ROLAP*** (Procesamiento Analítico en Línea Relacional):

Criterio / Ventaja	Justificación Técnica
Eficiencia de recursos (RAM y CPU)	Las arquitecturas analíticas tradicionales (como MOLAP) requieren motores adicionales que consumen mucha memoria. Al usar PostgreSQL para todo el proceso, centralizamos el esfuerzo en un motor robusto y altamente optimizable. Además, al diseñar una única tabla consolidada, PostgreSQL gestiona los índices de forma mucho más eficiente, evitando que múltiples tablas pequeñas compitan por la escasa memoria caché. A nivel de procesamiento, agrupar las métricas reduce drásticamente la carga de la CPU, ya que el sistema solo debe procesar una instrucción `INSERT` ligera en lugar de abrir múltiples transacciones simultáneas cada pocos segundos.
Optimización estricta del almacenamiento	En una máquina con poco espacio en disco, la redundancia de datos es nuestro mayor enemigo. El enfoque ROLAP en PostgreSQL almacena la información de forma compacta en tablas tradicionales, evitando la creación de costosos cubos multidimensionales. Al consolidar varias métricas (CPU, RAM, Disco) en una sola fila, evitamos repetir la columna de tiempo (`time`) —que ocupa bastante espacio— por cada métrica recolectada, logrando un ahorro de almacenamiento vital.
Garantía de atomicidad e integridad	PostgreSQL destaca por su cumplimiento estricto del modelo ACID. Al insertar todas las métricas de un ciclo de monitorización en una sola fila, garantizamos la atomicidad de los datos; es decir, aseguramos con total certeza que los valores de CPU, RAM y Disco corresponden exactamente al mismo instante de tiempo, sin desfases.
Sinergia natural con Grafana	El enfoque ROLAP encaja a la perfección con la forma en que Grafana consume los datos. Al delegar el esfuerzo analítico a PostgreSQL, podemos aprovechar funciones de agregación estándar de SQL (como `AVG`, `MAX` o `SUM`) directamente sobre las columnas de nuestra tabla de hechos. PostgreSQL resuelve estas consultas de forma nativa y rápida, entregando la información estructurada directamente a los paneles visuales.

En conclusión, nos hemos decantado por este gestor debido a las limitaciones de nuestra máquina y a la facilidad de escalabilidad.

2.4 Lenguaje de programación del script

Aquí es donde más nos hemos tenido que detener, puesto que nos ha costado mucho decidirnos por qué lenguaje emplear.
Hemos descartado Java, porque pese a que sea en el que más soltura tenemos debido a la carrera, es un lenguaje en el que no estamos cómodos para este tipo de actividades porque no hemos encontrado una librería que nos ahorre el trabajo.
Después nos hemos planteado usar C, puesto que debido al *hardware* y *specs* con los que contamos nos parecía el más idóneo, además, al estar en la mención de TI, este es el lenguaje por excelencia, pero debido a los consejos de Adrián y a que no hemos encontrado una biblioteca adaptada para esto, decidimos descartarlo.
Por último, el ganador de esta selección, Python, el cual es una navaja suiza, con muchas librerías adaptadas para nuestra práctica, precisamente por ese trabajo que nos podemos llevamos a ahorrar y pese a que sea el más pesado de los 3 lenguajes barajados, ha sido el ganador.
Adicionalmente para poder trabajar de una manera adecuada y no tener que instalarnos en local un gestor de versiones de Python, hemos creado un repositorio de GitHub para emplear UV y emplear la misma versión de Python y dependencias que las que posee la máquina virtual.

2.5 Creación del usuario para la bbdd

Usuario base de datos llamado **grafana**, contraseña **akelarre**
Siguiendo el consejo del profesor, hemos determinado la siguiente configuración para permitir al usuario **grafana** la lectura porque es el único permiso necesario.

```
-- 1. Permiso para conectar a la base de datos
GRANT CONNECT ON DATABASE metricas TO grafana;

-- 2. Permiso para entrar al esquema public (donde estan las tablas)
GRANT USAGE ON SCHEMA public TO grafana;

-- 3. Permiso para ver los datos de las tablas actuales
GRANT SELECT ON ALL TABLES IN SCHEMA public TO grafana;

```

Listing 5: Permisos de lectura para el *user* grafana en Postgres

2.6 Elección de las métricas

Una vez ya tenemos el usuario de grafana en la bbdd nos hemos centrado en qué metricas vamos a recoger en la misma, para que aporten información relevante sobre nuestro sistema.
A continuación incluimos la lista de las métricas y los motivos que nos han llevado a elegirlas..

1. **Métricas de CPU**
   - **Uso de CPU por núcleo:** porcentaje de utilización de la capacidad de procesamiento desglosado de forma individual para cada núcleo lógico (vCPU).
     - *Motivo:* nos permite detectar desbalanceos de carga en el sistema, donde un solo hilo satura un núcleo mientras los demás están inactivos.
   - **Tiempo en modo usuario, sistema e idle:** clasificación del tiempo de la CPU según si está ejecutando código de aplicaciones (*user*), procesos del propio kernel (*system*) o si está inactiva (*idle*).
     - *Motivo:* nos facilita el diagnóstico al diferenciar si el alto consumo es causado por las aplicaciones desplegadas o por una sobrecarga en la gestión del sistema operativo.
   - ***Load average* a 1, 5 y 15 minutos:** el promedio del número de procesos que están en la cola de ejecución o esperando recursos de E/S durante esos intervalos de tiempo.
     - *Motivo:* ofrece una visión temporal y realista del nivel de estrés y saturación del sistema, siendo mucho más representativo que los picos instantáneos de CPU.
2. **Métricas de Memoria**
   - **Memoria libre, usada, *buffers* y caché:** la distribución del total de la memoria usada(RAM disponible entre los datos activos de las aplicaciones), libre(sin asignar) y la *buffers/cache*(memoria utilizada por el *kernel*para acelerar el acceso a disco).
     - *Motivo:* puesto que es fundamental para evitar falsas alarmas, permitiendo discernir qué memoria está comprometida y cuál podría liberarse si una aplicación lo necesitara.
   - **Uso de *swap*:** lantidad de espacio en el disco duro que el sistema operativo está utilizando como memoria virtual temporal.
     - *Motivo:* al estar en una VM con una RAM limitada, nos sirve para prevenir caídas drásticas de rendimiento por depender del disco en lugar de la memoria física puesto que es mucho más lento.
   - **(*Page faults*):** el número de veces que un proceso requiere acceder a un bloque de memoria que ha sido paginado al disco y no se encuentra en la RAM activa.
     - *Motivo:* son un indicador directo de alta presión de memoria (*thrashing*), lo que degrada severamente el rendimiento general.
3. **Métricas de Disco**

- ***IOPS* (Operaciones de lectura/escritura por segundo):** el número total de transacciones individuales de lectura y escritura que el disco físico o virtual es capaz de completar en un segundo.
  - *Motivo:* permite evaluar si el almacenamiento subyacente soporta el volumen de carga transaccional, especialmente crítico en bases de datos.
- ***I/O Wait*:** porcentaje de tiempo que la CPU permanece inactiva porque está esperando a que el sistema de almacenamiento complete una operación.
  - *Motivo:* Detecta instantáneamente cuellos de botella generados por discos lentos o almacenamientos en red congestionados.
- **Tasa de lectura y escritura (*Bytes*/s):** volumen de datos brutos que fluyen desde y hacia el disco por unidad de tiempo.
  - *Motivo:* determina si las operaciones están limitadas por el ancho de banda del canal de almacenamiento.
- **Uso por partición:** porcentaje de espacio ocupado en cada sistema de archivos o punto de montaje independiente.
  - *Motivo:* Podemos prevenir caídas catastróficas de los servicios u operaciones del kernel por agotamiento de espacio en particiones críticas (`/var` o `/`).

4. **Métricas de Red**

- **Paquetes enviados y recibidos (vs. *bytes*):** tasa de unidades de datos de red gestionadas por la interfaz, independientemente de su tamaño en *bytes*.
  - *Motivo:* permite medir la carga transaccional del enrutador virtual y del *hypervisor*, ya que procesar muchos paquetes pequeños consume más CPU que pocos paquetes grandes.
- **Errores de transmisión y paquetes descartados (*drops*):** el recuento de paquetes que llegaron corruptos o que el sistema tuvo que descartar porque sus colas de procesamiento estaban llenas.
  - *Motivo:* diagnostica problemas de congestión, saturación de la interfaz de red o errores físicos en la capa de virtualización.
- **Conexiones activas TCP:** la cantidad de sesiones de red abiertas y mantenidas de forma concurrente con otros equipos o clientes.
  - *Motivo:* ayuda a predecir topes de escalabilidad de los servicios alojados antes de que afecten a los usuarios finales.

5. **Métricas de Procesos**

- **Estado de los procesos (*runnable, sleeping, zombie*):** conteo estadístico de los procesos según su estado actual: ejecutándose, esperando un evento, o "muertos"pero aún presentes en la tabla del sistema.
  - *Motivo:* crucial para identificar bloqueos de aplicaciones, fugas de recursos o mala gestión en la finalización de los hilos.
- ***Top* N procesos por uso de CPU / Memoria:** identificación de los servicios o aplicaciones específicas que están consumiendo la mayor cantidad de recursos en un momento dado.
  - *Motivo:* proporciona la causa raíz inmediata ante cualquier alerta de degradación de rendimiento.
- **Tiempo de CPU acumulado por proceso:** sumatorio de todo el tiempo de procesamiento que un hilo o aplicación ha consumido desde que se inició.
  - *Motivo:* nos permite cazar tareas en segundo plano ineficientes o bucles que degradan el sistema lentamente a lo largo del tiempo.

6. **Métricas de Sistema Avanzadas**

- ***Uptime* del sistema:** tiempo ininterrumpido que la máquina virtual lleva encendida y operativa desde su último reinicio.
  - *Motivo:* es un indicador básico de estabilidad y disponibilidad del servicio a largo plazo.
- **Cambios de contexto por segundo:** tasa a la cual el procesador guarda el estado de un proceso y carga el de otro para ejecutar múltiples tareas a la vez.
  - *Motivo:* tasas excesivamente altas indican ineficiencia y fragmentación, ya que la CPU gasta más tiempo cambiando entre tareas que ejecutándolas.
- **Interrupciones por segundo:** frecuencia de señales de hardware o software que obligan a la CPU a pausar su trabajo actual para atender un evento prioritario.
  - *Motivo:* útil para detectar anomalías en controladores, interfaces de red virtuales sobrecargadas o problemas a nivel del hipervisor.
- **Temperatura de CPU :** medición térmica del procesador subyacente.
  - *Motivo:* permite prever ralentizaciones inducidas por protección térmica (*thermal throttling*), aunque suele ser una métrica más propia del servidor físico (*bare-metal*).
- **Número de descriptores de fichero abiertos:**cantidad de archivos, *sockets* de red y otros recursos de I/O que los procesos tienen abiertos simultáneamente.
  - *Motivo:* Protege al sistema de bloqueos, ya que Linux tiene límites estrictos; superar este límite impide abrir nuevas conexiones o leer archivos.
- **Uso de la base de datos SQL (tamaño y conexiones):**espacio consumido por los archivos de la BD y número de consultas concurrentes que está manejando el motor (en nuestro caso PostgreSQL)
  - *Motivo:* supervisa directamente la salud del servicio que habitualmente concentra los mayores cuellos de botella de memoria y disco.

3. Base de datos en Postgressql

La base de datos que se ha realizado en Postgressql deberá recoger las métricas elegidas, y teniendo en cuenta las limitaciones de espacio se presentaban 3 opciones; tablas 'estrechas', donde cada métrica tendría su propia tabla, la tabla ancha, donde en cada instante de tiempo se tomarían todas las métricas y la solución mixta, agrupaciones de métricas en tablas anchas seccionadas.
La *tabla estrecha* presentaba el problema de la repetición constante del dato de tiempo, donde para un mismo instante en el que se capturasen varias metricas, para cada tabla se guardaría ese mismo instante de tiempo, resultando en datos redundantes y desaprovechamiento de espacio, aunque mayor manejo de los intervalos de cada captura y consultas más rápidas.
La opción de la *tabla ancha* nos permitía aprovechar ese mismo instante de tiempo y eliminar completamente la redundancia de datos, pero resultaría en consultas muy poco eficientes y tablas difíciles de manejar.
Finalmente, la solución que se eligió fue la **solución mixta**, que juntaba las características que nos interesaban de ambas propuestas aprovechando el espacio reduciendo la redundacia de los instantes de tiempo sin perder completamente la manejabilidad de los intervalos de captura.

3.1 Agrupación de datos y estructura de tablas

Como ya se ha explicado, las métricas serán agrupadas según cada cuánto tiempo serán capturadas, ya que hay métricas que tendrán poco cambio en el tiempo y por tanto no necesitarán capturarse continuamente y otras que será interesante observar sus cambios con mayor precisión.

Tabla	Frecuencia	Métricas incluidas
`perf_fast`	**10 - 15s**	CPU (por núcleo, load, times), RAM (usada, swap, page faults), Red (bytes/paquetes).
`sys_state`	**60s**	Conexiones TCP, Procesos por estado, Descriptores de fichero, Uptime, Tamaño DB SQL.
`disk_slow`	**5 min**	Uso por partición (GB/ %), IOPS, Wait time, Inodos.
`top_proc`	**60s**	Nombre, %CPU y %Mem del Top 3 (en tabla estrecha aparte).

Tabla de rendimiento rápido - metrics_perf

Frecuencia de captura: 10-15 segundos.

Contiene las métricas que fluctúan rápidamente y actúa como una visión global del sistema. Guarda el promedio de uso del procesador (cargas y porcentajes), el estado de la memoria RAM y Swap, y el tráfico de red de entrada/salida.
Al estar en la misma tabla, se pueden comparar fácilmente si un pico de de alguna métrica coincide con un pico de otra, como podría ser CPU y RAM.

```
    CREATE TABLE IF NOT EXISTS metrics_perf (
        time TIMESTAMPTZ(0) DEFAULT CURRENT_TIMESTAMP,

        -- CPU
        load_1min FLOAT, load_5min FLOAT, load_15min FLOAT,
        cpu_user_pct FLOAT, cpu_sys_pct FLOAT, cpu_idle_pct FLOAT,
        ctx_switches_sec FLOAT, interrupts_sec FLOAT,

        -- Memoria
```

```
        ram_used_pct FLOAT, ram_free_mb FLOAT,
        ram_cache_mb FLOAT, ram_buffers_mb FLOAT,
        swap_used_pct FLOAT, page_faults_sec FLOAT,

        -- Red instantánea
        net_rx_kbps FLOAT, net_tx_kbps FLOAT,
        net_err_in INT, net_err_out INT,
        PRIMARY KEY (time)
) PARTITION BY RANGE (time);
```

Listing 6: Tabla de Rendimiento Rápido

Tabla de CPU por Núcleo - metrics_cpu_cores

Frecuencia de captura: 10-15 segundos.

Se separa la visión global del rendimiento rápido con la visión separada por núcleos. Guarda el identificador del núcleo y su porcentaje de uso.
De esta forma se puede detectar si hay desbalances de carga y ajustarse a diferentes configuraciones de números de núcleos usados.

```
CREATE TABLE IF NOT EXISTS metrics_cpu_cores (
    time TIMESTAMPTZ(0) NOT NULL,
    core_id INT NOT NULL,
    usage_pct FLOAT NOT NULL,
    PRIMARY KEY (time, core_id)
) PARTITION BY RANGE (time);
```

Listing 7: Tabla de CPU por Núcleo

Tabla de Estado y Sistema - metrics_system_state

Frecuencia de captura: 1 minuto.

Registra métricas más descriptivas o que monitorizan la propia salud del sistema. Específicamente, guarda el tiempo que lleva encendida la máquina, el recuento de procesos (en ejecución, durmiendo, o zombies) las conexiones de red activas y, el tamaño que está ocupando la base de datos en el disco.

```
CREATE TABLE IF NOT EXISTS metrics_system_state (
    time TIMESTAMPTZ(0) DEFAULT CURRENT_TIMESTAMP,
    uptime_days FLOAT,
    procs_running INT, procs_sleeping INT, procs_zombie INT,
    tcp_conns_active INT,
    open_fds INT,
    db_size_mb FLOAT,
    db_active_conns INT,
    PRIMARY KEY (time)
) PARTITION BY RANGE (time);
```

Listing 8: Tabla de Estado y Sistema

Tabla de Almacenamiento y Discos - metrics_storage

Frecuencia de captura: 5 minutos.

Controla el estado físico de los discos, guardando para cada punto de montaje el porcentaje de espacio

usado, el espacio libre y la carga de operaciones de lectura/escritura (IOPS) junto con los tiempos de espera de la CPU.

```sql
CREATE TABLE IF NOT EXISTS metrics_storage (
    time TIMESTAMPTZ(0) NOT NULL,
    mount_point VARCHAR(50) NOT NULL,
    used_pct FLOAT,
    free_gb FLOAT,
    iops_read FLOAT, iops_write FLOAT,
    io_wait_ms FLOAT,
    PRIMARY KEY (time, mount_point)
) PARTITION BY RANGE (time);
```

Listing 9: Tabla de Almacenamiento y Discos

Tabla de Procesos Top - metrics_top_procs

Frecuencia de captura: 1 minuto.

Captura los procesos más pesados del sistema, funcionando como un ranking que guarda el nombre y el consumo exacto de CPU y Memoria de los procesos que más recursos están gastando en ese segundo, permitiendo saber quién esta produciendo los picos que se capturen en la tabla de rendimiento rápido

```sql
CREATE TABLE IF NOT EXISTS metrics_top_procs (
    time TIMESTAMPTZ(0) NOT NULL,
    rank INT, -- 1, 2, 3...
    proc_name VARCHAR(100),
    cpu_pct FLOAT,
    mem_pct FLOAT,
    PRIMARY KEY (time, rank)
) PARTITION BY RANGE (time);
```

Listing 10: Tabla de Procesos Top

3.2 Gestión del almacenamiento y particionamiento

Una vez establecida la estructura, se presentaba el problema del almacenamiento (capturas cada poco tiempo y crecimiento infinito de los datos), que acabaría consumiendo el espacio total de la máquina y afectaría a las consultas a nuestra base de datos, colapsando nuestra máquina si no teníamos cuidado.

Como solución y para que el sistema se mantenga autónomo y eficiente, las tablas han sido particionadas de forma dinámica en rangos de tiempo como se podía observar en las estructuras de las tablas con la característica de PostgreSQL con `PARTITION BY RANGE (time)`
Así, en vez de tener una tabla gigante, estas actúan como tabla padre y estructura base, y los datos realmente se guardarán en las tablas hijas correspondientes al día de la captura del dato.

3.2.1 Automatización del proceso

Dada la naturaleza de los datos que se van a capturar, con tener la información de un día entero o dos será suficiente para dar la visión necesaria sobre la evolución del sistema ante diferentes cargas.
De esta forma, se ha creado una función en nuestra base de datos para automatizar el proceso y eliminar los datos antiguos. Esta función se ejecutará una vez al día, y se encargará de crear las nuevas particiones (mañana) y borrar las anteriores (anteayer).
Así en nuestro sistema mantendremos siempre 3 tablas hijas de cada tabla padre, la de ayer que ya está completa, la de hoy que será donde se guarden los datos capturados y la de mañana, vacía y preparada para el próximo día.

Así conseguimos una base de datos predecible en tamaño, constante y actualizada y eficiente tanto en consultas como en almacenamiento.

3.3 Explicación de los scripts sobre SQL

Para que todo lo anteriormente explicado funcione, se han creado una serie de scripts para la máquina virtual para la creación y automatización del proceso, que se explicarán en mayor detalle:

- **create_tables.sql**:
  Crea la estructura de las tablas 'padre' explicadas en Agrupación de datos y estructura de tablas, junto a una tabla *default* por si ocurre algún problema en la creación de las particiones por día
- **create_partitions.sql**:
  Crea las particiones de hoy y mañana, necesario antes de automatizar el proceso de particionamiento y borrado.
- **rotate_partitions.sql**:
  La principal encargada de la automatización de aprovisionamiento y limpieza, crea la función en la base de datos `manage_all_metrics_partitions`. Esta función ejecutará para cada tabla 'padre':
  1. Calcular fecha de tabla a borrar (anteayer, dos días de antiguedad) y la fecha de la tabla a crear (mañana, un día a futuro)
  2. Crear la tabla física 'hija' del futuro para guardar los datos reales
  3. Eliminar la tabla física 'hija' antigua de anteayer para limpiar datos innecesarios
- **ejecutar_rotacion.sh**:
  Script para ejecutar con crontab que llamará a la función `manage_all_metrics_partitions` de nuestra base de datos

4. Conexión con grafana

Una vez que tenemos todos configurado y los scripts han pasado un par de días recopilando datos, procedemos a conectar todo con el grafana para mostrar los gráficos.

4.1 Permisos en PostgreSQL para el usuario `grafana`

Para solucionar los errores de acceso y configurar correctamente al usuario dentro de la base de datos `metrics`, fue necesario acceder como administrador e ingresar los siguientes comandos SQL:

4.1.1 Acceso a la base de datos

```
sudo -u postgres psql -d metrics

```

4.1.2 Permisos de Lectura (Para el Dashboard de Grafana)

Estos permisos son los ideales para que la herramienta visual pueda consultar los datos sin riesgo de modificarlos:

```
GRANT CONNECT ON DATABASE metrics TO grafana;
GRANT USAGE ON SCHEMA public TO grafana;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO grafana;

```

4.1.3 Permisos de Escritura (Para el Script de Python)

Estos permisos permiten que el script recolector pueda guardar (insertar) las métricas recolectadas en las tablas:

```
GRANT INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO grafana;

```

5. Convertir el Script en un Daemon (Servicio de Systemd)

Para evitar que el recolector se detenga al cerrar la terminal y asegurar que se ejecute en segundo plano de forma ininterrumpida, se configuró como un servicio nativo de Linux mediante `systemd`.

5.1 Creación del archivo de configuración

Se creó el archivo de servicio utilizando el editor `nano`:

```
sudo nano /etc/systemd/system/monitor_sistema.service

```

Dentro de este archivo, se introdujo la siguiente configuración:

```
[Unit]
Description=Monitor de Metricas del Sistema en Python
After=network.target postgresql.service

[Service]
User=usuario
WorkingDirectory=/home/usuario/ScriptsPython
ExecStart=/usr/bin/python3 /home/usuario/ScriptsPython/main.py
```

```
Restart=always
RestartSec=5
Environment=PYTHONUNBUFFERED=1

[Install]
WantedBy=multi-user.target

```

5.2 Activación y ejecución del servicio

Finalmente, se ejecutaron los siguientes comandos para recargar los servicios de Systemd, habilitar el inicio automático con el sistema y arrancar el daemon en ese preciso instante:

```
sudo systemctl daemon-reload
sudo systemctl enable monitor_sistema.service
sudo systemctl start monitor_sistema.service

```

Nota extra: Para comprobar el estado del servicio en cualquier momento, se puede utilizar el comando:

```
sudo systemctl status monitor_sistema.service

```

6. Códigos Python

6.1 Módulo de recolección de métricas (Collectors)

El sistema implementa un conjunto de *collectors* encargados de recopilar información del estado y rendimiento de la máquina. Estos componentes constituyen la base del pipeline de monitorización, ya que generan los datos que posteriormente serán almacenados en bases de datos y visualizados mediante Grafana.

Cada *collector* se especializa en un ámbito concreto del sistema (CPU, memoria, red, almacenamiento, procesos, etc.), siguiendo un diseño modular que favorece la escalabilidad y mantenibilidad.

6.1.1 Diseño general

Todos los *collectors* comparten una serie de características comunes:

- Obtienen un *timestamp* unificado para garantizar coherencia temporal en los datos.
- Delegan la obtención de métricas en módulos especializados (`metrics.*`).
- Transforman los datos en estructuras homogéneas (diccionarios o listas de diccionarios).
- No realizan persistencia, respetando la separación de responsabilidades.

Este enfoque sigue principios de bajo acoplamiento y alta cohesión, permitiéndonos modificar o extender cada collector de forma independiente.

6.1.2 Collector de uso de CPU por núcleo

El collector de CPU por núcleo tiene como objetivo obtener el porcentaje de uso de cada uno de los núcleos del procesador en un instante determinado. Este componente trabaja con métricas de alta granularidad, lo que permite analizar el comportamiento individual de cada núcleo, especialmente en sistemas multinúcleo donde la carga puede distribuirse de forma desigual.

Para facilitar su almacenamiento y posterior análisis en bases de datos de series temporales, hemos optado por representar cada núcleo como una entidad independiente en lugar de agruparlos en una única estructura. Este es el funcionamiento del collector:

- Obtiene un timestamp común para todas las mediciones.
- Recupera el uso de CPU por núcleo mediante el módulo de métricas.
- Recorre la lista de núcleos y construye una estructura de datos con:
  - Identificador del núcleo.
  - Porcentaje de uso.
  - Marca temporal.

El resultado es una lista de registros independientes, uno por cada núcleo.

6.1.3 Collector de métricas de rendimiento global

Este collector agrupa métricas de alta frecuencia relacionadas con CPU, memoria y red, proporcionando una visión global del rendimiento del sistema. Se trata del componente más completo del sistema de monitorización, ya que integra múltiples fuentes de datos en una única estructura, lo que nos permite reducir el número de operaciones de almacenamiento y mejorar la eficiencia global del sistema.

Incorpora además el cálculo de métricas derivadas a partir de diferencias entre estados previos y actuales, permitiendo obtener tasas por segundo en indicadores como interrupciones, cambios de contexto o tráfico de red, lo cual resulta fundamental para el análisis dinámico del sistema. Este es el funcionamiento del collector:

- Obtención de métricas de CPU:
  - Carga media del sistema.
  - Porcentaje de uso (usuario, sistema, inactivo).
  - Cambios de contexto por segundo.
  - Interrupciones por segundo.
- Obtención de métricas de memoria:
  - Uso de RAM.
  - Memoria libre, cache y buffers.
  - Uso de swap.
  - Fallos de página por segundo.
- Obtención de métricas de red:
  - Tráfico entrante y saliente.
  - Errores de red.

6.1.4 Collector de procesos más relevantes

Este collector obtiene información sobre los procesos más consumidores de recursos del sistema con el objetivo de identificar aquellos que tienen mayor impacto en el rendimiento. Esto nos permite detectar posibles cuellos de botella o comportamientos anómalos en la ejecución del sistema.

Para facilitar su visualización, especialmente en herramientas como Grafana, introducimos un sistema de ranking que ordena los procesos según su consumo de CPU y memoria. Además, el número de procesos analizados es configurable, lo que nos aporta flexibilidad y adaptabilidad a distintos escenarios de monitorización. Este es el funcionamiento del collector:

- Obtiene el timestamp actual.
- Recupera los N procesos más relevantes según uso de CPU y memoria.
- Genera una lista ordenada donde cada elemento contiene:
  - Posición en el ranking.
  - Nombre del proceso.
  - Uso de CPU.
  - Uso de memoria.

6.1.5 Collector de almacenamiento

Este collector recopila información sobre el estado del almacenamiento del sistema, combinando métricas de ocupación de disco con estadísticas de entrada y salida. Esta aproximación nos permite no solo conocer el nivel de uso de los discos, sino también analizar su actividad, lo cual es clave para detectar problemas de rendimiento asociados al subsistema de almacenamiento.

La información se organiza por punto de montaje, facilitando la correlación entre uso y actividad del disco. Contemplamos valores por defecto en caso de ausencia de datos de entrada/salida, lo que incrementa la robustez del sistema ante posibles inconsistencias. Este es el funcionamiento del collector:

- Obtiene el estado de uso de cada punto de montaje.
- Calcula estadísticas de I/O a partir de estados previos.
- Combina ambas fuentes de datos en una estructura unificada.

Para cada punto de montaje se generan métricas como:

- Porcentaje de uso del disco.
- Espacio libre disponible.
- Operaciones de lectura y escritura por segundo.
- Tiempo de espera de operaciones de I/O.

6.1.6 Collector de estado del sistema

Este collector recopila métricas de baja frecuencia relacionadas con el estado general del sistema, proporcionando una visión global que complementa las métricas de rendimiento. A diferencia de otros collectors orientados a capturar variaciones rápidas, este componente se centra en información estructural, lo que nos permite interpretar correctamente el comportamiento del sistema en su conjunto.

Por este motivo, su ejecución se realiza con menor frecuencia, reduciendo la carga sobre la máquina monitorizada sin perder información relevante. Su papel es especialmente importante en el diagnóstico, ya que nos aporta contexto a las métricas obtenidas por otros collectors. Este es el funcionamiento del collector:

- Tiempo de actividad del sistema.
- Número de procesos en diferentes estados.
- Conexiones TCP activas.
- Descriptores de archivo abiertos.
- Estadísticas de base de datos (tamaño y conexiones activas).

6.2 Módulo de obtención de métricas (Metrics)

El módulo de métricas constituye la capa encargada de interactuar directamente con el sistema operativo para obtener información en bruto sobre el estado y rendimiento de la máquina.

A diferencia de los *collectors*, estos componentes no realizan transformación estructural compleja ni agregación, sino que se centran en la obtención eficiente de datos mediante llamadas a la biblioteca `psutil` y otras utilidades del sistema.

6.2.1 Diseño general

Las funciones de este módulo presentan las siguientes características:

- Acceso directo a recursos del sistema operativo.
- Obtención de métricas en bruto o ligeramente procesadas.
- Separación entre contadores acumulativos y métricas derivadas.
- Uso de timestamps para permitir cálculos de tasas.

Este diseño nos permite reutilizar estas funciones en distintos collectors, favoreciendo la modularidad del sistema.

6.2.2 Métricas de CPU

El módulo de métricas de CPU proporciona funciones para obtener información tanto instantánea como acumulativa del procesador. Este módulo permite analizar distintos aspectos del comportamiento de la CPU, incluyendo la carga media del sistema, el uso porcentual en sus distintos estados (usuario, sistema e inactivo), el uso por núcleo y eventos del sistema como cambios de contexto e interrupciones.

Hemos diseñado este módulo separando la obtención de contadores acumulativos del cálculo de métricas derivadas, lo que nos aporta flexibilidad y nos permite reutilizar los datos en distintas capas del sistema. El módulo se apoya en la biblioteca `psutil` para obtener información del sistema:

- **Carga media**: obtenida mediante funciones del sistema operativo.
- **Uso de CPU**: calculado como porcentaje de tiempo en distintos estados.
- **Uso por núcleo**: permite analizar la distribución de carga.

Además, incluimos funciones que calculan tasas por segundo a partir de contadores acumulativos, como:

- Cambios de contexto por segundo.
- Interrupciones por segundo.

Para ello, proporcionamos acceso a contadores acumulativos junto con un timestamp, lo que permite calcular métricas derivadas en capas superiores.

6.2.3 Métricas de memoria

Este módulo proporciona información sobre el uso de memoria RAM y swap, así como métricas relacionadas con el sistema de memoria virtual. Se recogen indicadores como el porcentaje de uso de memoria, la memoria disponible, la memoria utilizada como cache y buffers, así como el uso de swap.

El diseño nos permite calcular métricas derivadas como los fallos de página por segundo a partir de diferencias entre estados, facilitando el análisis dinámico del comportamiento de la memoria. Para mejorar la eficiencia, hemos optado por devolver múltiples valores relacionados en una única llamada, reduciendo el número de accesos al sistema. Las métricas se obtienen mediante llamadas a `psutil`, transformando los valores a unidades más manejables como megabytes. Adicionalmente, calculamos:

- Número de fallos de página por segundo, a partir de diferencias entre estados.

Para ello, proporcionamos contadores acumulativos junto con un timestamp que permiten realizar estos cálculos.

6.2.4 Métricas de red

El módulo de red se encarga de medir el tráfico y los errores asociados a las interfaces de red. Las métricas principales incluyen el tráfico entrante y saliente, así como los errores de red. El diseño se basa en el uso de contadores acumulativos de bytes transmitidos y recibidos, a partir de los cuales calculamos tasas de transferencia normalizadas por el intervalo temporal.

Esta separación entre obtención de contadores y cálculo de métricas nos permite reutilizar la información en distintos contextos y mejorar la flexibilidad del sistema. El throughput de red se calcula a partir de la diferencia entre bytes enviados y recibidos en dos instantes de tiempo, normalizado por el intervalo temporal. Esto permite obtener:

- Kilobytes por segundo recibidos.
- Kilobytes por segundo enviados.

También proporcionamos acceso directo a contadores acumulativos de red, incluyendo bytes transmitidos y recibidos, junto con un timestamp.

6.2.5 Métricas de almacenamiento

Este módulo recopila información sobre el uso de disco y estadísticas de entrada/salida, permitiéndonos analizar tanto la ocupación del sistema de ficheros como la actividad del subsistema de almacenamiento. Se consideran métricas como el porcentaje de uso del disco, el espacio libre disponible y las operaciones de lectura y escritura por segundo. La información se estructura por disco o punto de montaje, facilitando su análisis y visualización.

Incorporamos mecanismos de gestión de errores para evitar interrupciones cuando no es posible acceder a determinadas particiones, aumentando la robustez del sistema. Para el uso de disco:

- Recorremos todas las particiones del sistema.
- Obtenemos el porcentaje de uso y espacio libre.

Para las estadísticas de I/O:

- Calculamos operaciones de lectura y escritura por segundo.
- Obtenemos el tiempo de ocupación del disco.

6.2.6 Métricas de procesos

Este módulo permite analizar el comportamiento de los procesos en ejecución en el sistema, proporcionándonos información tanto sobre su estado como sobre su consumo de recursos. Incluye funcionalidades para contar procesos según su estado y para identificar aquellos que presentan un mayor uso de CPU y memoria.

Hemos priorizado la robustez del sistema mediante el uso de mecanismos de control de excepciones, evitando fallos derivados de procesos que finalizan durante la iteración o restricciones de acceso. Asimismo, limitamos la información recogida a los atributos esenciales, reduciendo la sobrecarga del sistema. El sistema recorre la lista de procesos activos y obtiene información relevante como:

- Estado del proceso.

- Uso de CPU.
- Uso de memoria.

Implementamos mecanismos de control de excepciones para evitar errores durante la iteración.

6.2.7 Métricas del sistema

Este módulo proporciona información general sobre el estado global del sistema y recursos asociados, permitiéndonos complementar las métricas de rendimiento con una visión más estructural. Incluye indicadores como el tiempo de actividad del sistema, el número de conexiones TCP activas, los descriptores de archivo abiertos y estadísticas de la base de datos. Para ello, combinamos distintas fuentes de información, incluyendo la biblioteca `psutil`, el sistema de archivos y la ejecución de comandos externos.

Además, contemplamos posibles fallos en estas operaciones, devolviendo valores por defecto en caso de error para garantizar la estabilidad del sistema. El módulo combina diferentes fuentes de información:

- `psutil` para métricas del sistema operativo.
- Acceso al sistema de archivos para descriptores abiertos.
- Ejecución de comandos externos para obtener estadísticas de base de datos.

Realizamos consultas a la base de datos para obtener:

- Tamaño de la base de datos.
- Número de conexiones activas.

6.3 Módulo de persistencia de datos

Este módulo es el encargado de gestionar la interacción con la base de datos, constituyendo la última etapa del pipeline de monitorización. Su responsabilidad principal es almacenar de forma estructurada las métricas previamente recolectadas, garantizando la integridad y disponibilidad de los datos para su posterior análisis y visualización en Grafana.

Hemos diseñado este módulo separando claramente la gestión de conexiones de la lógica de escritura, lo que nos permite mejorar la mantenibilidad del sistema y facilitar posibles cambios en la infraestructura de persistencia. Además, utilizamos consultas parametrizadas para evitar problemas de seguridad y mejorar la robustez del sistema.

6.3.1 Gestión de conexiones

Este componente se encarga de establecer y gestionar las conexiones con la base de datos PostgreSQL. Para ello, utilizamos tanto conexiones individuales como un pool de conexiones, lo que nos permite optimizar el uso de recursos y mejorar el rendimiento en escenarios de alta concurrencia.

Hemos incorporado el uso de un pool de conexiones para reutilizar conexiones existentes en lugar de crear nuevas constantemente, reduciendo así el coste asociado a la conexión con la base de datos. Asimismo, proporcionamos funciones específicas para obtener y liberar conexiones, lo que nos permite controlar de forma explícita su ciclo de vida. El módulo proporciona las siguientes funcionalidades:

- Creación de una conexión individual a la base de datos mediante los parámetros de configuración.
- Creación de un pool de conexiones con un número mínimo y máximo de conexiones.
- Obtención de una conexión desde el pool para su uso.
- Liberación de una conexión, devolviéndola al pool para su reutilización.

6.3.2 Inserción de métricas

Este componente es responsable de almacenar en la base de datos las métricas generadas por los distintos collectors. Para ello, definimos funciones específicas para cada tipo de métrica, lo que nos permite mantener una estructura clara y alineada con el modelo de datos.

Hemos diseñado estas funciones de manera que cada una se encargue de una tabla concreta, facilitando la organización de los datos y su posterior consulta. Además, utilizamos inserciones en lote cuando es posible, lo que mejora significativamente el rendimiento al reducir el número de operaciones sobre la base de datos. También gestionamos explícitamente las transacciones, aplicando commit en caso de éxito y rollback en caso de error, lo que nos permite garantizar la integridad de los datos. El proceso de inserción sigue el siguiente esquema general:

- Construimos la sentencia SQL de inserción correspondiente a la tabla destino.
- Extraemos los valores necesarios desde las estructuras de datos generadas por los collectors.
- Ejecutamos la inserción mediante un cursor de base de datos.
- Confirmamos la operación mediante commit en caso de éxito.
- En caso de error, realizamos rollback para evitar inconsistencias.

Dependiendo del tipo de métrica, distinguimos dos casos:

- Inserciones individuales, utilizadas para métricas agregadas como rendimiento global o estado del sistema.
- Inserciones múltiples mediante operaciones en lote, utilizadas para métricas repetitivas como uso por núcleo, almacenamiento o procesos.

Este enfoque nos permite equilibrar claridad, eficiencia y robustez en el proceso de persistencia de datos.

6.4 Módulo de configuración

El módulo de configuración centraliza todos los parámetros necesarios para el funcionamiento del sistema de monitorización, permitiéndonos definir de forma unificada aspectos relacionados con la base de datos, los intervalos de recolección, el comportamiento del sistema y opciones de depuración.

Hemos diseñado este módulo con el objetivo de desacoplar la configuración del resto de la lógica de la aplicación, lo que nos permite modificar el comportamiento del sistema sin necesidad de alterar el código de los distintos componentes. Este enfoque mejora la mantenibilidad, facilita la adaptación a distintos entornos y simplifica el proceso de despliegue.

Además, agrupamos los parámetros por categorías funcionales, lo que nos permite organizar la configuración de forma clara y facilitar su comprensión. Entre estas categorías se incluyen la configuración de la base de datos, los intervalos de ejecución de los collectors, la configuración de procesos, parámetros del sistema y opciones de depuración.

6.4.1 Funcionamiento

El módulo define un conjunto de constantes que son utilizadas por los distintos componentes del sistema:

- Parámetros de conexión a la base de datos, incluyendo host, puerto, nombre de la base de datos, usuario y contraseña.
- Intervalos de ejecución para cada tipo de collector, permitiendo controlar la frecuencia de recolección de métricas según su naturaleza.

- Número de procesos a monitorizar en el collector de procesos, lo que permite ajustar el nivel de detalle del análisis.
- Parámetros del scheduler, como el tiempo de espera entre iteraciones del bucle principal.
- Opciones de depuración, que permiten activar o desactivar la salida de información para diagnóstico.

Estos valores son importados por los distintos módulos del sistema, lo que nos permite mantener una configuración coherente y centralizada en toda la aplicación.

6.5 Módulo de planificación (Scheduler)

El módulo de planificación es el encargado de coordinar la ejecución de los distintos collectors y gestionar el flujo de datos desde la obtención de métricas hasta su almacenamiento en la base de datos. Este componente actúa como el núcleo del sistema, controlando cuándo y con qué frecuencia se ejecuta cada tipo de recolección.

Hemos diseñado el scheduler como un bucle continuo que evalúa periódicamente el tiempo transcurrido para decidir qué collectors deben ejecutarse en cada iteración. Este enfoque nos permite gestionar múltiples frecuencias de muestreo dentro de un único proceso, optimizando el uso de recursos y evitando la necesidad de múltiples hilos o procesos concurrentes.

Además, utilizamos un sistema de timestamps para cada tipo de métrica, lo que nos permite controlar de forma precisa los intervalos de ejecución. También integramos la obtención de estados previos y actuales para el cálculo de métricas derivadas, garantizando la coherencia en los datos obtenidos. Por último, incorporamos mecanismos de control de errores independientes para cada bloque de ejecución, lo que nos permite mantener el funcionamiento del sistema incluso ante fallos puntuales.

6.5.1 Funcionamiento

El scheduler sigue un ciclo de ejecución continuo basado en los siguientes pasos:

- Inicializamos variables de control temporal para cada tipo de collector, permitiéndonos gestionar sus intervalos de ejecución.
- Obtenemos un estado inicial del sistema a partir de contadores acumulativos de CPU, memoria, red y disco, necesario para el cálculo de métricas derivadas.
- En cada iteración del bucle:
  - Calculamos el tiempo actual.
  - Obtenemos un nuevo estado del sistema.
- Evaluamos cada tipo de métrica en función de su intervalo configurado:
  - Ejecutamos los collectors de rendimiento y CPU por núcleo en intervalos cortos.
  - Ejecutamos el collector de estado del sistema en intervalos medios.
  - Ejecutamos el collector de almacenamiento en intervalos más largos.
  - Ejecutamos el collector de procesos según su frecuencia configurada.
- Para cada ejecución:
  - Recogemos las métricas mediante el collector correspondiente.
  - Insertamos los datos en la base de datos utilizando el módulo de persistencia.
  - Gestionamos posibles errores mediante bloques de control independientes.
- Actualizamos el estado previo con el estado actual para la siguiente iteración.
- Introducimos una pausa controlada en cada iteración del bucle para evitar un uso excesivo de CPU.

Este diseño nos permite coordinar de forma eficiente la recolección y almacenamiento de métricas, manteniendo un equilibrio entre precisión temporal y consumo de recursos.

6.6 Módulo principal de ejecución

El módulo principal constituye el punto de entrada del sistema de monitorización, siendo el encargado de inicializar los recursos necesarios y poner en marcha el flujo completo de ejecución. Su responsabilidad principal es establecer la conexión con la base de datos y lanzar el scheduler, que coordina la recolección y almacenamiento de métricas.

Hemos diseñado este componente con una estructura sencilla y clara, delegando la mayor parte de la lógica en módulos especializados. Este enfoque nos permite mantener una separación adecuada de responsabilidades, facilitando la mantenibilidad del sistema y evitando concentrar lógica compleja en el punto de entrada.

Además, incorporamos mecanismos de gestión de errores a nivel global, lo que nos permite capturar fallos críticos durante la ejecución y garantizar el cierre adecuado de los recursos utilizados, especialmente la conexión a la base de datos.

6.6.1 Funcionamiento

El flujo de ejecución del módulo principal es el siguiente:

- Inicializamos la aplicación mostrando un mensaje de arranque del sistema.
- Establecemos una conexión con la base de datos utilizando el módulo de conexión.
- Lanzamos el scheduler, que se encarga de ejecutar los collectors y almacenar las métricas de forma continua.
- En caso de que se produzca un error durante la ejecución, capturamos la excepción y mostramos un mensaje de error.
- Finalmente, garantizamos el cierre de la conexión a la base de datos para liberar los recursos utilizados.

Este diseño nos permite iniciar el sistema de forma controlada y asegurar que todos los componentes se ejecutan correctamente desde un único punto de entrada.

7. Métricas de Grafana

Una vez que hemos configurado todo el sistema con los *scripts* y las bbdd toca la parte más divertida que es ver los resultados de nuestro trabajo en grafana2.2.

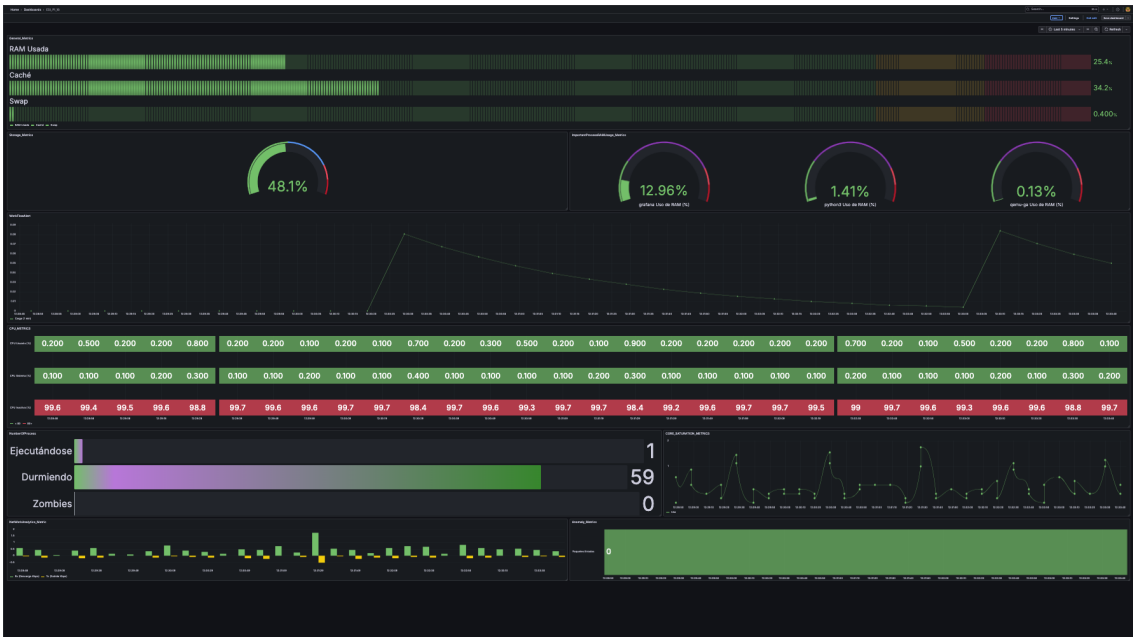


Figura 1: *Dashboard* de grafana con todos los paneles que hemos considerado importantes

Ahora vamos a ir panel a panel explicando cada métrica, el motivo de la misma y l *query* empleado en caso de ser necesario.

7.1 Memoria

La motivación para este panel es la limitación en cuanto a los recursos que tiene nuestra máquina, ya que esta sólo cuenta con 2GB de memoria.
Estaremos consultando las columnas `ram_used_pct`, `ram_cache_mb`, `swap_used_pct` y `page_faults_sec` en la tabla `metrics_perf`.

7.1.1 SWAP-CACHÉ-RAM

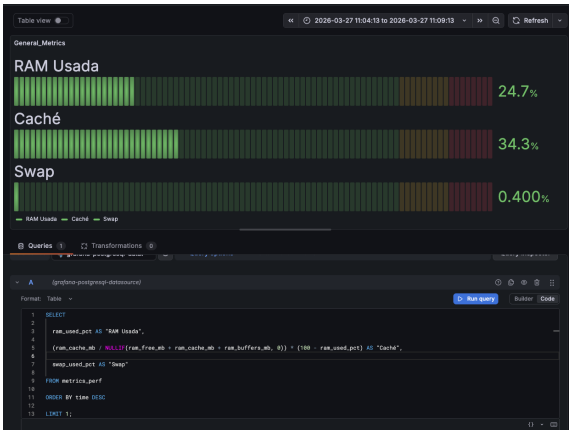


Figura 2: Información sobre el uso general de la memoria RAM en nuestra máquina virtual

La consulta es bastante sencilla, la RAM y la SWAP la sacamos directamente de la la consulta, pero la caché al estar en MB la hemos tenido que convertir a %, por lo que hemos empleado una regla de tres para primero sacar el tamaño de la caché, multiplicarlo por la RAM para conseguir así el tamaño que ocupa (asegurándonos que no pueda dar *null* porque rompería el gráfico). Hemos decidido emplear este tipo de gráficos por estética y para tener de un vistazo los tres "tipos"de memoria.

7.1.2 Métrica de la RAM que consumen ciertos procesos

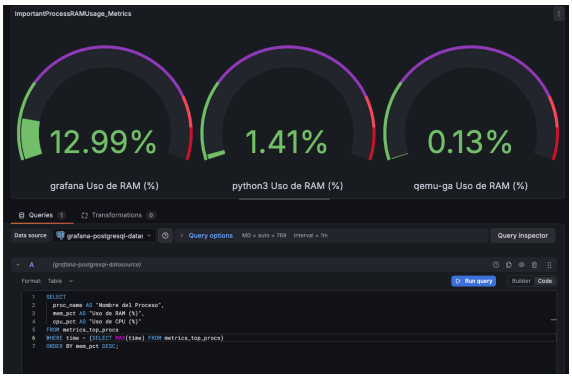


Figura 3: Procesos que más recursos de memoria RAM están consumiendo

Al igual que la anterior, esta consulta es muy simple, conseguimos el nombre del proceso, el uso de ram y el uso de CPU de los procesos que más recursos están consumiendo de la máquina.
Este panel es clave porque llegado el momento, podemos hacer un *pkill -9* al proceso y evitar que la máquina colapse.

7.2 Almacenamiento

Se registran los valores de `used_pct`, `io_wait_ms`, `iops_read` e `iops_write` en la tabla `metrics_storage`.

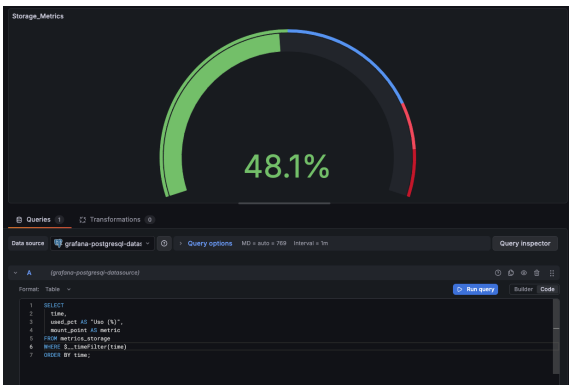


Figura 4: % de espacio en el disco cupado.

Esta métrica es seguramente la más importante de todas según los avisos de Adrián, puesto que al estar recolectando datos durante elevados periodos de tiempo podemos llegar a perder el acceso a la máquina, como ya les ha pasado a compañeros del año pasado.
Aunque Edgar ha programado la bbdd para que vaya eliminando la tabla de los días pasados (para una explicación más detallada consultar 3.2), lo que nos debería asegurar que la bbdd no se coma el espacio en el disco, hemos decidido tener este panel por si hubiese que meterse a al VM a liberar espacio en el disco a mano.

7.3 CPU

Las métricas de CPU nos son muy útiles para detectar tareas muy pesadas que puedan crear cuellos de botellas que ralenticen el sistema, además, nos permite comprobar si algún daemon es sospechoso, puesto que si de la nada un demonio o un proceso de usuario empieza a consumir mucha CPU podemos ponerlo en cuarentena e investigar qué está haciendo en nuestro sistema o directamente eliminarlo.
En general hemos empleado las columnas `cpu_user_pct`, `cpu_sys_pct`, `cpu_idle_pct` y los promedios de carga (`load_Xmin`) de la tabla `metrics_perf`, junto con `usage_pct` por núcleo en `metrics_cpu_cores`.

7.3.1 Uso de la CPU

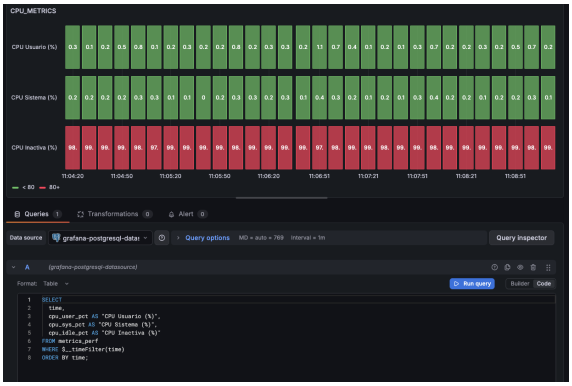


Figura 5: % de uso de la CPU de los usuarios, el sistema y la disponible.

Hemos decidido emplear un *status history* para tener una manera visual de cómo evoluciona el uso de CPU en función de los procesos de usuario y del sistema en la escala de tanto por cien.

7.3.2 Uso de los *cores*

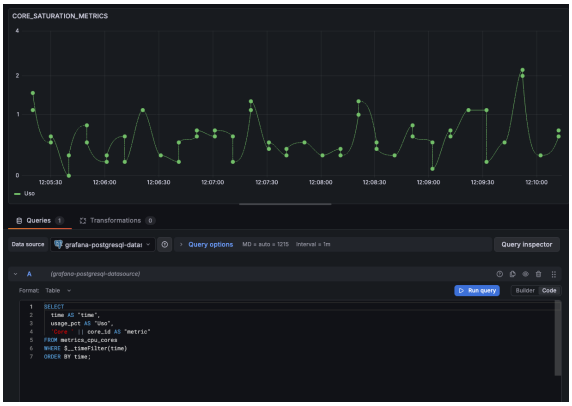


Figura 6: Evolución de los usos de los *cores* de nuestra CPU.

La motivación de esta métrica no es muy común, puesto que se basa en medir el uso de los *cores* (en el caso de nuestra máquina de los dos núcleos con los que cuenta). Esto es muy útil si nos centramos en paralelizar procesos, cosa que es muy útil en caso de tener que realizar tareas muy pesadas en cuanto al uso del procesador en un entorno tan limitado como el nuestro. Por lo que esta información nos resulta útil por dos motivos, el primero es el más evidente, comprobar si bajo una carga de trabajo nuestra CPU se está viendo superada y la segunda es comprobar lo bien paralelizado que tenemos un código, puesto que podemos ver qué tanto nos estamos acercando al 100 % del uso de los núcleos.

7.4 Procesos

Se emplean métricas como `db_size_mb`, `procs_zombie`, `open_fds` y los datos de la tabla `metrics_top_procs`.

7.4.1 Número y tipos ejecutándose

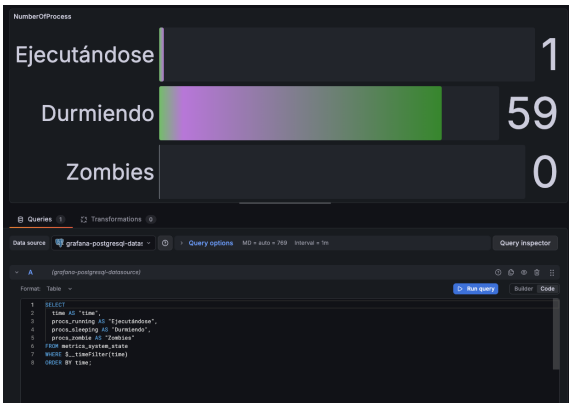


Figura 7: Tipos de procesos y cantidad de los mismos en la máquina

Esta métrica podría parecer poco útil, pero todo lo contrario, el saber qué tipos de procesos se están ejecutando aporta una información muy útil al administrador del sistema, para empezar porque tiene un conteo de todo lo que se está ejecutando en su sistema y adicionalmente puede comprobar si esos procesos *zombies* no son en realiadad *malware* al igual que con los procesos que se están ejecutando.
Este panel del grafana, es muy útil en un caso real, puesto que al estar en un entorno con características algo precarias (en comparación a los sistemas actuales a los que estamos acostumbrados) es vital tener monitoreados los procesos que corren en el sistema, ya que uno sólo que se desmadre puede provocar un colapso.
Sobre la *query*, al igual que en la gran mayoría, es una consulta muy sencilla, donde accedemos a una tabla y monstramos los datos en función del tiempo.

7.5 *Network*

Estamos en TI, por lo que no podían faltar las métricas sobre el uso de red que está haciendo nuestra máquina.
En concreto estaremos consultando las columnas `net_rx_kbps`, `net_tx_kbps`, `net_err_in` y `tcp_conns_active`.

7.5.1 Tráfico entrante y saliente

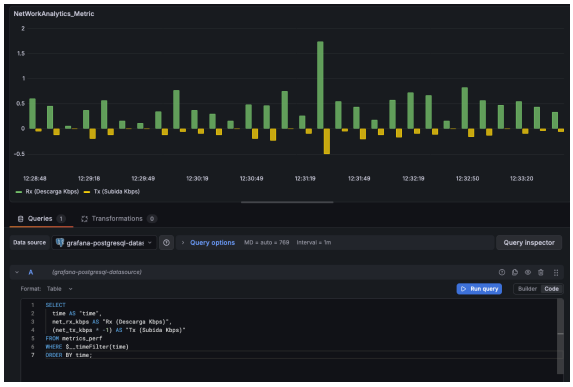


Figura 8: Análisis del tráfico en Kbps entrante y saliente a nuestra VM.

Métrica muy interesante, ya que podemos tener un seguimimiento en tiempo real del tamaño del tráfico que circula por nuestra red, esto es muy útil por diversos motivos, el primero es por ver cuánto volumen de datos le estamos enviando a grafana, y el segundo es que nos permite detectar cuando hay mucha actividad en nuestra red y en caso de ser una actividad sospechosa, podemos intuir que estamos bajo un ataque o que hemos sido infectados de alguna manera, ya que si juntamos un proceso zombie extraño, con un alto uso de la cpu y un pico de tráfico entrante, el administrador de tarea puede deducir que algo raro está ocurriendo.

7.5.2 Paquetes perdidos

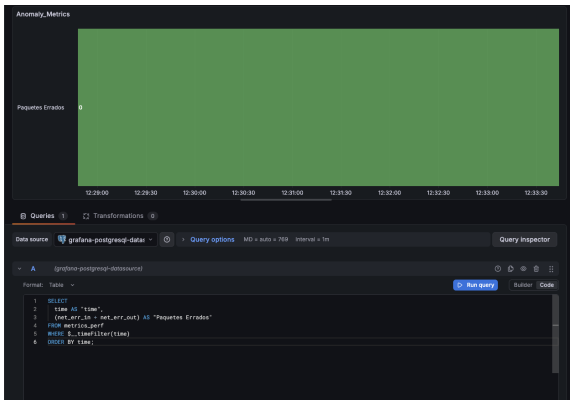


Figura 9: Análisis del número de paquetes de red perdidos.

Decidimos meter esta métrica puesto que es muy común que trabajemos con git o sistemas de controles de versiones o con un código que pueda modificar la tarjeta de red de nuestra máquina, por lo que el saber el número de paquetes que hemos perdido puede servirnos para saber si la modificaciones que hemos hecho ha dejado a la máquina sin internet o si hay algún problema con el servidor contra el que nos queremos conectar.

8. Reparto del trabajo

Nombre	**% del Trabajo**	**Responsabilidades**
Marcos Álvarez Fulconis	33 %	Diseño y programación de los *scipts* en Python, redacción de estas tareas en el documento
Edgar Santiesteban Mendo	33 %	Diseño de la bbdd y creación de las tablas, redacción de esta parte en el documento
Álvaro López Primo	33 %	Gestión de la máquina virtual y de los *daemons*, enlace con el grafana,repositorio de git, creación del entorno virutal y redacción de estas partes en el documento.

Cuadro 1: Distribución del trabajo y responsabilidades del equipo.